

MEMORIA DE PROYECTO FINAL DE GRADO

BAME

ASADOR DE POLLO · MURCIA

Plataforma de Pedidos Online

Desarrollo full-stack de una plataforma digital de pedidos y gestión para la cadena de asadores BAME en la Región de Murcia: frontend Angular 21, backend Spring Boot 3 y pasarela de pagos Stripe.

Angular 21

Spring Boot 3

Java 21

PostgreSQL

Stripe Payments

JWT Auth

Tailwind CSS v4

i18n · 5 idiomas

AUTOR

Natxo Pérez

TUTOR

Fernando Domingo Ruiz

CENTRO EDUCATIVO

IES José Planes, Espinardo (Murcia) · CFGS DAW · Junio 2026

Página intencionalmente en blanco.

Licencia y autoría

Esta memoria ha sido elaborada como Trabajo de Fin de Grado del Ciclo Formativo de Grado Superior en **Desarrollo de Aplicaciones Web (DAW)**, impartido en el **IES José Planes** de Espinardo (Murcia), durante el curso 2025–2026.

Desarrollado íntegramente por **Natxo Pérez** bajo tutorización de **Fernando Domingo Ruiz**. El código fuente se distribuye bajo licencia **MIT**. Este documento se publica bajo **Creative Commons BY–NC–SA 4.0**.



Creative Commons Atribución–NoComercial–CompartirIgual 4.0 Internacional

Puede reproducirse con atribución y sin fines comerciales, manteniendo la misma licencia.

Autor	Natxo Pérez
Tutor	Fernando Domingo Ruiz
Centro	IES José Planes, Espinardo (Murcia)
Ciclo	CFGS Desarrollo de Aplicaciones Web (DAW)
Curso	2025–2026 · Junio 2026
Proyecto	BAME — Plataforma de pedidos online para asador de pollo

Índice de contenidos

01  Resumen extendido

02  Palabras clave

03  Introducción

04  Estado del arte y trabajos relacionados

05  Análisis de objetivos y metodología

06  Diseño y resolución del trabajo realizado

07  Presupuesto

08  Conclusiones y vías futuras

09  Bibliografía y Webgrafía

Resumen extendido



BAME Asador de Pollo es una plataforma digital de pedidos online y gestión interna desarrollada íntegramente como Trabajo de Fin de Grado del Ciclo Formativo de Grado Superior de Desarrollo de Aplicaciones Web en el IES José Planes (Espinardo, Murcia). El proyecto nace de la necesidad real de digitalizar el proceso de pedidos de la cadena de asadores BAME, que hasta la fecha dependía de plataformas de terceros con comisiones de entre el 15 y el 35 % por pedido.

La plataforma cubre el ciclo completo de un pedido: el cliente navega la carta de productos segmentada por local, añade ítems al carrito, selecciona método de pago (tarjeta online vía Stripe o pago en local) y confirma el pedido. El sistema notifica al local en tiempo real y el responsable actualiza el estado hasta su entrega o recogida. Incluye además un módulo de reservas de mesas con calendario interactivo y confirmación automática por correo electrónico.

Técnicamente se divide en dos capas. El **frontend** está construido con **Angular 21** en modo standalone components y usa la API de Signals para gestión reactiva del estado, estilizado con **Tailwind CSS v4** mediante tokens de diseño propios. El **backend** es una API REST con **Spring Boot 3** y **Java 21**, conectada a **PostgreSQL** en Supabase. Autenticación mediante **JWT** con cuatro roles: CLIENTE, RESPONSABLE, GERENTE y ADMIN. Despliegue en **Vercel** (frontend) y **Render** (backend).

La plataforma incorpora internacionalización en **cinco idiomas** —español, inglés, francés, alemán y árabe con soporte RTL completo—, menú de accesibilidad con ajustes de tamaño de fuente, contraste y reducción de movimiento, y diseño responsivo con soporte para modo claro y oscuro.

4 Roles de usuario	5 Idiomas	9 Módulos backend	2 Métodos de pago	~280h Desarrollo total
------------------------------	---------------------	-----------------------------	-----------------------------	----------------------------------

Palabras clave

02

 Angular 21

 Spring Boot 3

 PostgreSQL

 JWT Authentication

 Stripe Payments

 Pedidos online

 i18n

 Accesibilidad

 Responsive Design

 Tailwind CSS v4

 Reservas de mesa

 Gestión multilocal

 REST API

 Dark / Light Mode

Introducción

03

Contexto y motivación

La digitalización de la restauración ha crecido exponencialmente desde la pandemia de COVID-19, acelerando la adopción de soluciones de pedido online. Sin embargo, la mayoría de las soluciones disponibles están orientadas a grandes cadenas o a plataformas agregadoras (Glovo, JustEat, Uber Eats) que cobran comisiones de entre el 15 % y el 35 % por pedido, lo que resulta inasumible a largo plazo para negocios medianos.

BAME Asador de Pollo es una cadena familiar con varios locales en la Región de Murcia especializada en pollo asado y productos de temporada. La necesidad de ofrecer un canal propio de pedido online sin depender de intermediarios es el origen de este proyecto: el desarrollo de una plataforma web completa con marca propia y control total sobre la experiencia del usuario.

Objetivo principal

Diseñar, desarrollar y desplegar una aplicación web full-stack que permita a los clientes realizar pedidos de recogida en local y reservas de mesa a través de internet, y a los trabajadores y gestores administrar pedidos, carta, horarios y locales desde un panel interno.

Alcance del proyecto: Desde el registro y autenticación de clientes hasta el pago online con Stripe, incluyendo la gestión completa del ciclo de vida de un pedido y un sistema de reservas con confirmación automática por email.

Estructura de la memoria

Esta memoria se organiza en nueve secciones. Tras el capítulo introductorio se analiza el estado del arte (§4), se describen los objetivos y la metodología (§5), se detalla el diseño técnico (§6), se presenta el presupuesto (§7) y se recogen conclusiones y vías futuras (§8), seguidas de la bibliografía (§9).

Estado del arte y trabajos relacionados

04

Antes de diseñar BAME se analizaron las soluciones existentes para la gestión de pedidos y reservas en restauración, identificando cuatro categorías: plataformas agregadoras generalistas, plataformas de reservas especializadas, soluciones white-label para restaurantes y páginas web tradicionales.



Glovo / JustEat / Uber Eats

Plataformas agregadoras

Marketplaces de delivery y recogida que agregan múltiples restaurantes. Alta visibilidad, pero comisiones de 15–35 % por pedido y dependencia total del algoritmo. El restaurante pierde el control sobre su marca y los datos del cliente.

Alta comisión

Sin branding propio

Sin datos del cliente



TheFork (ElTenedor)

Reservas de restaurante

Plataforma líder en reservas de mesa en España. Permite gestionar disponibilidad y confirmaciones automáticas, pero no contempla pedidos de recogida ni carta digital. Modelo de suscripción mensual con coste fijo elevado.

Solo reservas

Suscripción mensual

Sin pedidos



Flipdish / Otter

White-label para restaurantes

SaaS con app y web de pedidos con marca propia. Incluyen integración con TPV y gestión de pedidos. Los planes de precio son elevados (desde 149 €/mes) y la personalización visual es limitada.

Coste mensual alto

Personalización limitada



WordPress + WooCommerce

Páginas web tradicionales

Alta flexibilidad inicial, pero mantenimiento costoso, rendimiento limitado y experiencia de usuario inferior a una SPA. Sin roles de trabajador ni gestión multilocal nativa.

Mantenimiento complejo

Sin multilocal

Diferenciación clave: BAME se posiciona como solución **propia y sin comisiones** que combina pedidos de recogida, reservas de mesa y gestión multilocal, con identidad de marca completa y coste operativo mínimo tras el despliegue inicial.

Tecnologías evaluadas

En frontend se analizaron **React 19** (Next.js), **Vue 3** y **Angular 21**. Se optó por Angular por su arquitectura estructurada, tipado estricto y la nueva API de Signals que elimina la necesidad de gestores de estado externos. En backend se evaluaron **Node.js + Express**, **NestJS** y **Spring Boot**. Se eligió Spring Boot por la robustez de su ecosistema (Spring Security, Spring Data JPA, Spring Mail) y la familiaridad con Java.

Análisis de objetivos y metodología

Objetivos funcionales

01

Catálogo de productos

Carta digital por local y categoría, con imágenes, precios y alérgenos.

02

Carrito y pedidos

Carrito persistente, resumen de pedido y flujo de confirmación.

03

Pagos con Stripe

Stripe Checkout para pago con tarjeta y confirmación por webhook.

04

Reservas de mesa

Calendario por local con confirmación automática por email.

05

Panel responsable

Gestión de pedidos con actualización de estado en tiempo real.

06

Panel gerente

Edición de carta, horarios y configuración del local asignado.

07

Panel administrador

Gestión global de locales, usuarios y datos de la plataforma.

08

Multiidioma (i18n)

Interfaz en 5 idiomas con detección automática y soporte RTL.

Objetivos no funcionales

- ✓ Carga inicial inferior a 2 s (lazy loading + bundle splitting)
- ✓ Diseño responsivo mobile-first, testeado desde 320 px de ancho
- ✓ Accesibilidad WCAG 2.1 AA: contraste mínimo 4,5:1, navegación por teclado, ARIA labels
- ✓ Seguridad: JWT con expiración configurable, CORS restringido, sanitización de inputs
- ✓ Arquitectura stateless escalable horizontalmente sin cambios de código

Metodología de desarrollo

Se siguió una metodología **iterativa e incremental** similar a Scrum, adaptada al trabajo individual. Sprints semanales con entregables concretos, GitHub Projects para el

seguimiento y ramas de funcionalidad por módulo.

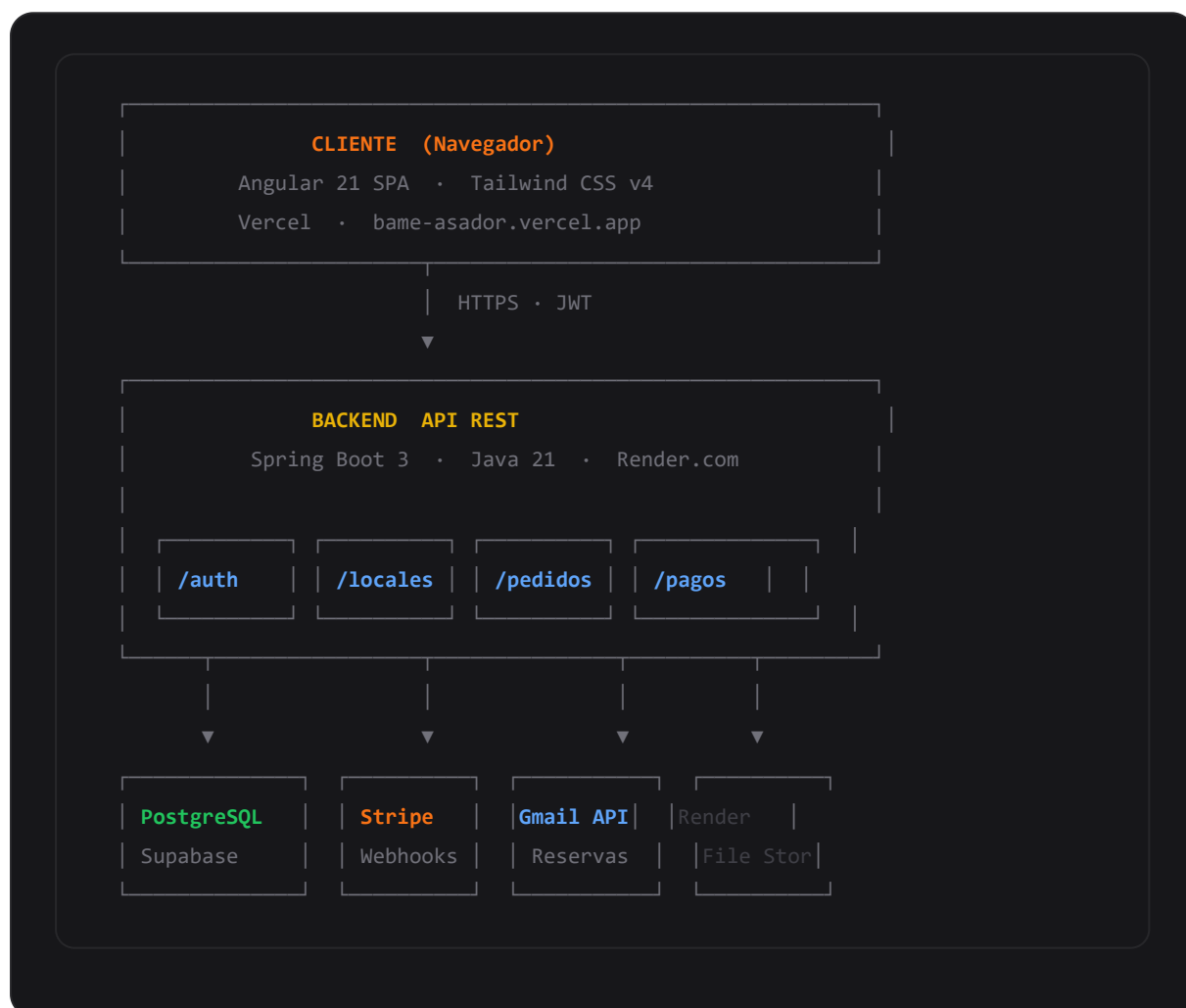
- 1 Análisis y diseño — semanas 1–2**
Levantamiento de requisitos con el cliente, definición de roles y casos de uso. Diseño de la base de datos y wireframes en Figma.
- 2 Backend núcleo — semanas 3–6**
Configuración Spring Boot, modelos JPA, autenticación JWT y endpoints de usuarios, locales y productos.
- 3 Frontend base — semanas 5–9**
Scaffolding Angular, tokens Tailwind, componentes shared, autenticación y vistas de carta y carrito.
- 4 Pagos y reservas — semanas 9–12**
Stripe Checkout, webhooks de confirmación, sistema de reservas con disponibilidad y notificaciones por email.
- 5 Paneles de gestión — semanas 12–15**
Panel responsable, panel gerente y panel administración con guards y controles de acceso por rol.
- 6 i18n, accesibilidad y despliegue — semanas 15–18**
Internacionalización en 5 idiomas, menú de accesibilidad, optimización de rendimiento y despliegue en Vercel + Render.

Diseño y resolución del trabajo realizado

06

Arquitectura del sistema

La arquitectura sigue el patrón cliente-servidor, con separación total entre la capa de presentación (SPA Angular) y la capa de negocio (API REST Spring Boot). Toda la comunicación se realiza mediante HTTPS con JWT en la cabecera `Authorization: Bearer <token>`.



Estructura del backend

PAQUETES — COM.BAME.API

```
com.bame.api/  
├─ auth/           // JWT filter, AuthController, tokens  
├─ usuarios/       // registro, perfil, gestión por ADMIN  
├─ locales/        // locales, horarios, disponibilidad  
├─ productos/     // carta, categorías, alérgenos  
├─ pedidos/       // creación, estados, historial  
├─ pagos/         // Stripe Checkout, webhooks  
├─ reservas/      // reservas, slots, confirmación email  
├─ contacto/      // formulario → Gmail API  
└─ common/        // excepciones, DTOs base, config CORS
```

Autenticación y roles

Cuatro roles con permisos acumulativos, gestionados con Spring Security + JWT:

 **CLIENTE**

 **RESPONSABLE**

 **GERENTE**

 **ADMIN**



CLIENTE: Navegar carta, hacer pedidos y gestionar sus reservas.



RESPONSABLE: Todo lo anterior + actualizar estados de pedidos del local asignado.



GERENTE: Todo lo anterior + editar carta, horarios y configuración del local.



ADMIN: Acceso completo a todos los locales, usuarios y datos de la plataforma.

ANGULAR — GUARD DE ROLES CON SIGNALS

export const

```
roleGuard = (roles: Role[]) => canActivateFn(): boolean | UrlTree => {
```

const

```
auth = inject(AuthService);
```

const

```
router = inject(Router);
```

const

```
user = auth.currentUser(); // Signal<User | null>
```

if

```
(!user || !roles.includes(user.role))
```

return

```
router.createUrlTree(['/home']);
```

return true

```
;  
});
```

Frontend — Angular 21

 **core/** — AuthService, CartService, I18nService, ThemeService, interceptors HTTP.

 **features/** — home, carta, reservas, checkout, auth, contact, admin, gerente.

 **shared/** — error-card, accessibility-menu, skeletons, tabler-icon wrapper.

 **ANGULAR — CARRITOSERVICE CON SIGNALS**

```
@Injectable({ providedIn: 'root' })
```

```
export class
```

```
CartService {
```

```
private
```

```
items = signal<CartItem[]>([]);
```

```
total = computed(() =>
```

```
this
```

```
.items().reduce((s, i) => s + i.precio * i.cantidad, 0)  
);
```

```
add(producto: Producto) {
```

```
this
```

```
.items.update(curr => {
```

```
const
```

```
found = curr.find(i => i.id === producto.id);
```

```
if
```

```
(found)
```

```
return
```

```
curr.map(i =>  
  i.id === producto.id ? { ...i, cantidad: i.cantidad + 1 } : i  
);
```

```
return
```

```
[...curr, { ...producto, cantidad: 1 }];  
});
```

```
}  
  
}
```

Integración de pagos — Stripe Checkout

El flujo sigue el patrón de Stripe Checkout Session: el frontend solicita al backend la creación de una sesión, el backend devuelve una URL de redirección a la página segura de Stripe, el cliente paga y Stripe notifica la confirmación mediante webhook.

</> SPRING BOOT — WEBHOOK STRIPE

```
@PostMapping("/webhook")
```

```
public
```

```
    ResponseEntity<String> handleWebhook(  
        @RequestBody String payload,  
        @RequestHeader("Stripe-Signature") String sig  
    ) {  
        Event ev = Webhook.constructEvent(payload, sig, secret);
```

```
        if
```

```
        ("checkout.session.completed".equals(ev.getType())) {  
            Session s = (Session) ev.getDataObjectDeserializer()  
                .getObject().orElseThrow();  
            pedidoService.confirmarPago(s.getMetadata().get("pedidoId"));  
        }
```

```
        return
```

```
        ResponseEntity.ok("OK");  
    }
```

Sistema de diseño y despliegue

Tokens propios en `@theme` de Tailwind CSS v4: naranja `#FF4500`, amarillo `#FFD700`, fondo oscuro `#0e0e10`. Fuentes: Bricolage Grotesque (headings) y Plus Jakarta Sans (cuerpo). El

menú de accesibilidad persiste ajustes de fuente, contraste y movimiento en `localStorage` como clases en `<html>`.

-  **Vercel** — Frontend: despliegue automático desde `main`, CDN global, HTTPS automático.
-  **Render** — Backend Spring Boot: contenedor Docker, escalado automático, env vars seguras.
-  **Supabase** — PostgreSQL gestionado con backups automáticos y pooling de conexiones.

Presupuesto

07

Desglose de horas por fase

FASE	DESCRIPCIÓN	HORAS
Análisis y diseño	Requisitos, diseño de BBDD, wireframes Figma y planificación	28
Backend — núcleo	Spring Boot, JPA, JWT, endpoints auth/usuarios/locales	42
Backend — funcional	Módulos productos, pedidos, reservas, pagos y contacto	38
Frontend — base	Angular scaffolding, tokens Tailwind, shared components, rutas	34
Frontend — features	Carta, carrito, checkout, reservas, contacto, historial, perfil	56
Paneles de gestión	Panel responsable, panel gerente y panel administrador	30
i18n y accesibilidad	Traducciones en 5 idiomas, menú de accesibilidad, soporte RTL	22
Testing e integración	Pruebas de integración, corrección de errores, ajustes UX	18
Despliegue y documentación	Configuración Vercel/Render, manual de usuario y esta memoria	14
Total de horas		282 h

Presupuesto económico

CONCEPTO	DETALLE	COSTE (€)
Desarrollo (282 h x 20 €/h)	Análisis, diseño, programación, testing y documentación	5.640,00
Vercel (frontend)	Plan Hobby gratuito para proyectos personales y TFG	0,00
Render (backend)	Plan Free — instancia con sleep tras 15 min de inactividad	0,00
Supabase (PostgreSQL)	Plan Free — 500 MB almacenamiento, 2 proyectos activos	0,00
Stripe (pagos)	Sin cuota mensual; comisión por transacción en modo live	0,00
Herramientas de desarrollo	IntelliJ IDEA (licencia estudiante), VS Code, Figma (plan gratuito)	0,00
Dominio — estimación año 1	bame-asador.es — tarifa orientativa registrador .es	9,95
Total presupuesto estimado		5.649,95 €

Nota: El coste real para el cliente es 0 € al tratarse de un TFG sin contraprestación económica. En producción real, el mantenimiento anual se estima en 800–1.200 €/año.

Conclusiones y vías futuras

Logros alcanzados

El proyecto BAME ha cumplido todos los objetivos funcionales y no funcionales definidos en la fase de análisis. Se ha desarrollado una plataforma full-stack completa, funcional y desplegada en producción.

- ✓ **Plataforma funcional en producción:** Accesible en Vercel con el backend operativo en Render y la base de datos en Supabase.
- ✓ **Pagos reales con Stripe:** Proceso seguro que cumple PCI DSS al no manejar datos de tarjeta directamente.
- ✓ **Sistema multirol completo:** Los cuatro roles funcionan con sus guards y paneles diferenciados.
- ✓ **Internacionalización en 5 idiomas:** Toda la interfaz, incluyendo mensajes de error y notificaciones, traducida en es/en/fr/de/ar.
- ✓ **Accesibilidad WCAG 2.1 AA:** Menú de accesibilidad, contraste correcto y navegación completa por teclado.
- ✓ **Código limpio y mantenible:** Separación de responsabilidades, Signals para estado reactivo y ausencia de comentarios redundantes.

Reflexión personal

Este proyecto ha supuesto un salto cualitativo respecto a los ejercicios del ciclo formativo. Llevar un producto desde el análisis inicial hasta el despliegue en producción, con un cliente real cuyos requisitos evolucionan, ha consolidado competencias más allá del código: gestión del tiempo, decisiones técnicas con restricciones reales y comunicación con perfiles no técnicos.

La elección de Angular 21 con la API de Signals fue inicialmente arriesgada por ser tecnología reciente, pero resultó la decisión correcta: el código es más limpio y predecible que con las alternativas basadas en RxJS puro consideradas al inicio.

Vías futuras



Puntos de fidelización

Acumular puntos por pedido y canjear por descuentos. Ya referenciado en la pantalla de login.



Notificaciones push

WebSockets o SSE para notificar cambios de estado del pedido sin refrescar la página.



App móvil

PWA mejorada o app nativa con Ionic/Capacitor para App Store y Google Play.



Analíticas

Dashboard para gerentes: ventas, productos más pedidos, horas pico y tasa de conversión.



Delivery propio

Módulo de reparto con asignación de repartidor y tracking de entrega en tiempo real.



Asistente IA

Recomendación de productos basada en historial de pedidos mediante embeddings ligeros.

Bibliografía y Webgrafía



Documentación oficial

- [1] Angular Team. *Angular Documentation v21*. Google, 2025. angular.dev
- [2] Spring Team. *Spring Boot Reference Documentation 3.x*. VMware, 2025. docs.spring.io/spring-boot
- [3] Tailwind Labs. *Tailwind CSS v4 Documentation*. 2025. tailwindcss.com/docs
- [4] Stripe, Inc. *Stripe Checkout Documentation*. 2025. stripe.com/docs/checkout
- [5] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. 2024. postgresql.org/docs/16/

Libros y artículos

- [6] Wieruch, R. *The Road to React*. Leanpub, 2024.
- [7] Walls, C. *Spring in Action, 6ª edición*. Manning Publications, 2022.
- [8] OWASP Foundation. *OWASP Top Ten 2021*. 2021. owasp.org/www-project-top-ten/
- [9] W3C. *Web Content Accessibility Guidelines (WCAG) 2.1*. W3C Recommendation, 2018.

Recursos y herramientas

- [10] JetBrains. *IntelliJ IDEA Community Edition*. JetBrains s.r.o, 2025.
- [11] Microsoft. *Visual Studio Code*. Microsoft Corporation, 2025.

[12] Figma Inc. *Figma Design Tool*. 2025.

[13] Vercel Inc. *Vercel Deployment Platform*. 2025. vercel.com/docs

[14] Render.com. *Render Cloud Application Platform*. 2025.

[15] Supabase, Inc. *Supabase — The Open Source Firebase Alternative*. 2025.

[16] Auth0. *Introduction to JSON Web Tokens*. jwt.io/introduction

[17] Bootstrap Icons contributors. *Bootstrap Icons v1.11*. The Bootstrap Authors, 2024.

[18] Google Fonts. *Bricolage Grotesque & Plus Jakarta Sans*. Google LLC, 2024.

BAME

ASADOR DE POLLO · MURCIA

Plataforma de Pedidos Online

Autor	Natxo Pérez
Tutor	Fernando Domingo Ruiz
Centro	IES José Planes, Espinardo
Ciclo	CFGS DAW
Fecha	Junio 2026